Scale-Aware Evolutionary Discovery of Wind-Farm Layout Optimizers with Large Language Models

Author One Author Two Author Three
ADDRESS
Correspondence: (EMAIL)

Abstract

We present a framework for the automated discovery of *deployable* learning-rate and constraint-penalty schedules for gradient-based wind-farm layout optimization using large language models (LLMs). Candidate schedules are expressed in a *scale-aware* parameterization in which the exploration learning rate is constructed from the rotor diameter and the only user-supplied scale is the constraint tolerance, so a single schedule generalizes across farms of different size with no free per-farm parameter. An island-based, quality-diversity evolutionary loop drives LLM coding agents to propose schedules, which a multi-stage *cascade* evaluator scores across a multi-farm, multi-scale training suite under a hard feasibility gate. A behavioral archive, content-addressed caching, and complete lineage provenance make every discovery auditable and, on a fixed platform, bit-reproducible; a firewall discipline keeps held-out and test performance out of the search. We describe the schedule interface, the fixed optimization skeleton, the evolutionary loop and cascade evaluator, the fitness, selection, and deployment rules, and the reproducibility and firewall machinery that together turn opportunistic program search into a repeatable engineering procedure.

1 Introduction

When designing a wind farm, the placement of turbines within the buildable area determines every downstream stage of a project, from land valuation to annual energy production (AEP). Because a fraction-of-a-percent gain in modeled AEP can translate into a large change in a project's financial case, layout optimization is a well-studied problem: it is a non-convex, non-linear, constrained optimization in which turbine positions interact through wake losses and must respect spacing and boundary (inclusion/exclusion zone) constraints.

Gradient-based solvers have become a de-facto method for this problem. The TopFarm stochastic gradient descent (SGD) optimizer (Quick et al., 2022) adapts the Adam algorithm (Kingma and Ba, 2015) with a scheduled learning rate and a scheduled linear constraint penalty. The *schedule* — how the learning rate, the penalty weight, and the two Adam moment-decay coefficients evolve over the course of the optimization — is a rich design space that governs the trade-off between exploring the layout and driving the final iterate onto the feasible set. It is precisely the kind of low-dimensional, high-leverage function that is difficult to design by hand and attractive to search automatically.

Large language models have recently been used as the mutation operator in evolutionary *program search*: FunSearch (Romera-Paredes et al., 2024) and AlphaEvolve (Google DeepMind, 2025) evolve populations of programs by repeatedly prompting an LLM to improve high-scoring candidates. We adopt this paradigm for optimizer-schedule design, and add three ingredients that make the discovered artifact *deployable* rather than merely high-scoring on one farm:

1. a **scale-aware schedule interface** (Section 2.2) that removes the free learning-rate parameter and builds the exploration scale from the rotor diameter, so a discovered schedule is scale-covariant and transfers across farms of different turbine size;
2. a **quality-diversity evolutionary loop with a cascade evaluator** (Sections 2.5–2.7) that scores candidates across a multi-farm, multi-scale training suite under a hard feasibility gate, rejecting infeasible or dominated schedules cheaply before spending on expensive high-count evaluations; and

3. a **provenance, firewall, and reproducibility layer** (Sections 2.9–4) — full lineage, a behavioral archive, content-addressed caching, a scoped mutator workspace, and a pre-registered selection and deployment criterion — so that any "the system discovered X" claim is auditable against what was seeded and what was measured.

The remainder of the paper describes the framework (Section 2), the wind farms and evaluation cells it is applied to (Section 3), and the reproducibility and cost-control machinery (Section 4).

2 Framework

This section describes the discovery framework: the optimization problem (2.1), the scale-aware schedule interface (2.2), the fixed optimization skeleton (2.3), the penalty normalization (2.4), the evolutionary loop (2.5) and its behavioral archive (2.6), the cascade evaluator (2.7), the fitness and feasibility rules (2.8), and the provenance, firewall, selection, and deployment machinery (2.9–2.11). Figure 1 gives the overall data flow.

2.1 Optimization problem

For a farm with N turbines at horizontal positions $(\boldsymbol{x}, \boldsymbol{y})$ we maximize the modeled annual energy production

$$\max_{\boldsymbol{x},\boldsymbol{y}} \; J(\boldsymbol{x}, \boldsymbol{y}), \qquad J = \frac{8760}{10^6} \sum_k w_k \, P\Big(\boldsymbol{x}, \boldsymbol{y};\, u_\infty^{(k)}, \theta^{(k)}\Big), \tag{1}$$

where P is the wind-farm power under inflow speed $u_\infty^{(k)}$ and direction $\theta^{(k)}$ with probability weight w_k, evaluated with an engineering wake model (Bastankhah and Porté-Agel, 2014), subject to the feasibility constraints

$$d(x_i, y_i) \geq 0 \;\; \forall i, \qquad \sqrt{(x_i - x_j)^2 + (y_i - y_j)^2} \;\geq\; D_{\min} \;\; \forall i \neq j, \tag{2}$$

where $d(x_i, y_i)$ is the signed distance from turbine i to the nearest inclusion-zone boundary (negative outside the buildable region) and $D_{\min}$ is the minimum inter-turbine spacing. Feasibility is judged against a tolerance $\gamma_{\min}$ (in metres): a layout is feasible when every turbine satisfies the boundary constraint to within $\gamma_{\min}$ and the pairwise spacing constraint. $\gamma_{\min}$ is the single user-supplied scale of the problem.

2.2 Scale-aware schedule interface

The object we search over is a pure function that returns, at each optimization step i, the four quantities the skeleton needs: a learning rate η_i, a constraint penalty multiplier α_i, and the two Adam moment-decay coefficients $\beta_{1,i}, \beta_{2,i}$:

$$\big(\eta_i, \alpha_i, \beta_{1,i}, \beta_{2,i}\big) = \mathcal{S}\big(i,\, T,\, D,\, D_{\min},\, N,\, \gamma_{\min},\, \alpha_0\big). \tag{3}$$

The arguments are the current step i, the total number of steps T, and the *problem-intrinsic scales*: the rotor diameter D, the minimum spacing $D_{\min}$, the turbine count N, the metre-valued constraint tolerance $\gamma_{\min}$, and a skeleton-computed penalty normalizer α_0 (Section 2.4). The Python signature is reproduced in Appendix A.

Two design choices make a discovered schedule deployable across scales:

Algorithm 1 Fixed optimization skeleton (the search controls only $\mathcal{S}$)

$\boldsymbol{s} \leftarrow$ wind-aware init$(D, D_{\min}, N, \text{resource})$; $\quad \boldsymbol{m}, \boldsymbol{v} \leftarrow \boldsymbol{0}$
$\alpha_0 \leftarrow \text{mean}\big(|\nabla_{\boldsymbol{s}} J(\boldsymbol{s})|\big)/D$; $\quad \alpha_0 \leftarrow \textsc{canonicalize}(\alpha_0)$ ▷ Section 2.4
for $i = 0, 1, \ldots, T-1$ **do**
  $(\eta_i, \alpha_i, \beta_{1,i}, \beta_{2,i}) \leftarrow \mathcal{S}(i, T, D, D_{\min}, N, \gamma_{\min}, \alpha_0)$
  $\boldsymbol{j} \leftarrow \nabla_{\boldsymbol{s}}\big[-J(\boldsymbol{s})\big] + \alpha_i \nabla_{\boldsymbol{s}} \gamma(\boldsymbol{s})$ ▷ objective + penalty gradient
  $\boldsymbol{m} \leftarrow \beta_{1,i}\boldsymbol{m} + (1-\beta_{1,i})\boldsymbol{j}$; $\quad \boldsymbol{v} \leftarrow \beta_{2,i}\boldsymbol{v} + (1-\beta_{2,i})\boldsymbol{j}^2$
  $\hat{\boldsymbol{m}} \leftarrow \boldsymbol{m}/(1-\beta_{1,i}^{i+1})$; $\quad \hat{\boldsymbol{v}} \leftarrow \boldsymbol{v}/(1-\beta_{2,i}^{i+1})$
  $\boldsymbol{s} \leftarrow \boldsymbol{s} - \eta_i \hat{\boldsymbol{m}}/\sqrt{\hat{\boldsymbol{v}}}$
end for
return $\boldsymbol{s}$

No free learning rate. There is no driver-supplied initial learning rate η_0 anywhere in Equation (3). The exploration scale must be constructed *inside* the schedule from the rotor diameter, e.g. a peak learning rate $\eta_{\text{peak}} = c\,D$ with c a schedule-internal constant or curve that the search may evolve but that is never a user knob. Because the turbine spacing, wake length, and feasible-region geometry all scale with D, expressing the exploration scale as a multiple of D makes the schedule scale-covariant: the same function applied to a farm with a different rotor diameter automatically takes appropriately sized steps.

Tolerance as the terminal learning rate. Following TopFarm's own convention, the terminal learning rate is a proxy for how tightly the boundary is ultimately satisfied; we expose it directly as the metre-valued tolerance $\gamma_{\min}$ toward which the schedule decays the learning rate. It is the only user-supplied scale, and a faithful schedule must respond to it: at a looser $\gamma_{\min}$ the schedule should stop decaying sooner, and its terminal behavior and feasibility should change accordingly (this responsiveness is checked in Section 2.7).

2.3 Fixed optimization skeleton

The schedule is the *only* thing the search controls. A fixed skeleton (Algorithm 1) handles initialization, gradient computation, and the Adam update. It (i) places the initial layout on a wind-direction-aware grid inside the buildable region (for the multi-zone case, on a zone-interior fill); (ii) forms the objective gradient of Equation (1) and the constraint gradient of the boundary and spacing penalties by automatic differentiation; (iii) combines them as $\boldsymbol{j} = \nabla_{\text{obj}} + \alpha_i \nabla_{\text{con}}$; and (iv) applies an Adam step with the scheduled $\eta_i, \beta_{1,i}, \beta_{2,i}$. The horizon is fixed at $T = 8000$ steps. Nothing in the skeleton contains a hardcoded learning rate; the entire step-size behavior comes from Equation (3).

2.4 Penalty normalization

So that the penalty multiplier is on a problem-intrinsic scale, the skeleton computes the normalizer

$$\alpha_0 = \frac{\text{mean}\left(|\nabla_{\boldsymbol{x}} J|,\ |\nabla_{\boldsymbol{y}} J|\right)}{D} \tag{4}$$

at the initial layout and passes it into Equation (3); the schedule composes the running penalty α_i from α_0. Normalizing the gradient magnitude by the rotor diameter (rather than by a learning rate) keeps the penalty scale covariant with the geometry and independent of the step-size choice, consistent with the removal of a free learning rate. To make evaluation bit-reproducible across *independent processes on a given platform*, α_0 is canonicalized to single precision at the skeleton boundary (CANONICALIZE in Algorithm 1): the double-precision gradient reduction agrees to far more significant figures than single precision, so rounding α_0 to a canonical single-precision value makes the same (schedule, cell, seed)

Algorithm 2 Evolutionary schedule discovery

seed each island archive with generation-0 ancestors (logged with lineage)
while cost < ceiling **do**
  for each island **do**
    $p \leftarrow$ SAMPLEPARENT(archive) ▷ fitness/novelty-aware
    materialize a scoped mutator workspace (Section 2.10)
    $c \leftarrow$ LLM-MUTATE(p, firewall-safe feedback)
    if NOVELTYFILTER rejects c **then continue**
    end if
    (fit, feasible, desc) $\leftarrow$ CASCADE(c) ▷ Algorithm/Section 2.7
    if feasible **then** ARCHIVEINSERT(island, c, fit, desc)
    end if
    LOGLINEAGE(c, p, engine, tokens, cost, desc, fit)
  end for
  periodically migrate elites between islands; checkpoint state
end while

reproduce identically in independent processes. This matters because the low–learning-rate tail of the optimization is sensitive to α_0. Across *different* hardware architectures a small floating-point drift remains; we therefore report cross-platform agreement to within a measured drift tolerance (Section 4) rather than bit-identity.

2.5 Evolutionary discovery loop

Schedules are discovered by a quality-diversity evolutionary loop (Algorithm 2), built on an island model with a MAP-Elites archive (Mouret and Clune, 2015) per island. Each iteration: (i) a parent schedule is sampled from an island's archive with a fitness- and novelty-aware sampler; (ii) an LLM mutation engine is prompted with the parent source and firewall-safe feedback and returns an improved schedule as Python source; (iii) a code-novelty filter rejects a child that is a near-duplicate of an already-seen program before any expensive evaluation is spent; (iv) the survivor is scored by the cascade evaluator (Section 2.7); and (v) it is inserted into the island's behavioral archive if it is elite for its cell. Islands periodically exchange elites. The loop runs until a token/cost ceiling is reached (Section 4).

The population is *seeded* with known-good schedules, logged as generation-0 ancestors with their port transforms (Section 2.9); the search is an engineering procedure and seeding is encouraged. The mutation operators are LLM coding agents accessed through a subscription/OAuth path (the Claude Agent SDK is the primary engine; a command-line Gemini engine and a deterministic mock engine are also supported). We adopt two eval-saving mechanisms from ShinkaEvolve (Sakana AI, 2025): code-novelty rejection-sampling and fitness/novelty-aware parent sampling; the island/MAP-Elites and cascade scaffolding follow OpenEvolve (OpenEvolve, 2025).

2.6 Behavioral archive

Each candidate is placed in a MAP-Elites archive keyed by four behavioral descriptors computed from the schedule function *before* evaluation, so that the archive covers qualitatively distinct schedule shapes rather than crowding a single family (Table 1): the peak learning rate relative to the rotor diameter, the terminal learning rate in metres, the coupling between the penalty and the learning rate, and the number of learning-rate restarts. Within each island, one elite is kept per occupied cell; feasibility dominates, then mean fitness, then a worst-cell tiebreak (Section 2.8). Seeding populates several descriptor cells from generation 0.

Table 1: Behavioral descriptors and their (frozen) bins for the MAP-Elites archive. Descriptors are computed from the schedule function prior to evaluation.

Descriptor	Bins
Peak learning rate / D	< 0.5, 0.5–0.8, 0.8–1.2, >1.2
Terminal learning rate (m)	≤ 0.01, 0.01–0.1, 0.1–1, >1
Penalty coupling	coupled, decoupled, cyclic
Learning-rate restarts	0, 1–2, ≥ 3

2.7 Cascade evaluator

Evaluating a schedule on every training cell at full seed count is expensive, so candidates pass through a cascade that spends compute in proportion to promise. Each stage runs the fixed skeleton (Algorithm 1) on a cell/seed and records the resulting AEP and feasibility.

Stage A (gross fast-reject). Two cheap cells × two seeds. This is a deliberately *gross* filter: a candidate is rejected only if any seed is infeasible or its AEP is more than about one percent below that cell's reference. It is *not* texture-floor-tight — a floor-tight Stage A would mass-reject the very exploration the quality-diversity archive exists to keep, so the texture floors are reserved for selection margins, not for this filter. The rejection rate by cause (infeasible vs. below-reference) is logged as a pilot diagnostic. No high-count cell is used here.

Stage B (training matrix). The full frozen training set (Table 2) × at least five *paired* seeds — the same seed produces the same wind-aware initial layout for the candidate and for the reference, so per-seed differences are paired. Each cell yields a percentage score (Section 2.8) and a candidate-feasibility flag; the cross-cell aggregate is the candidate's stage-B fitness.

Stage B$^+$ (elite tier). Top-k archive elites are re-scored on expensive high-count cells, where a longer per-evaluation budget is affordable. This stage runs only on dedicated compute and never inside the main loop. It also carries the *capability-frontier* cells — cells (a high-count single-polygon farm and a dense multi-zone farm under a unidirectional resource) on which even the reference schedule is infeasible. These are tracked at the elite tier as qualitative probes: a candidate that reaches strict feasibility there is a qualitatively new result. They never gate the per-generation search; off the dedicated compute they are simply marked pending and deferred.

Stage C (holdout & responsiveness). Elites are scored on the held-out farm for margin-aware selection (Section 2.11) and are re-scored at a looser tolerance to confirm the schedule *responds* to $\gamma_{\min}$ (feasibility and terminal behavior must change); a schedule invariant to the tolerance is rejected regardless of AEP. The held-out AEP never leaves this stage: only feasibility booleans and a margin-over-floor boolean cross the firewall (Section 2.10).

2.8 Fitness and feasibility

For a training cell c, over the shared paired seed set, the score is the percentage improvement in mean AEP over a reference schedule,

$$\text{score}_c \;=\; 100 \times \frac{\overline{\text{AEP}}_{\text{cand},c} - \overline{\text{AEP}}_{\text{ref},c}}{\overline{\text{AEP}}_{\text{ref},c}}, \tag{5}$$

where the reference is the native TopFarm monotone schedule at a rotor-diameter-scaled learning rate, $\eta_{\text{peak}} = c_0 D$ with c_0 calibrated once on the training farm, decaying to $\gamma_{\min}$. Expressing fitness as percent-over-reference makes cells of very different magnitude (thousands of GWh for the large offshore farms, hundreds for the small onshore multi-zone farm) commensurable. The reference AEP

$\overline{\text{AEP}}_{\text{ref},c}$ is used purely as a *scale constant*: it normalizes the candidate even on a cell where the reference itself is infeasible, and the reference's feasibility never confers credit.

Feasibility at $\gamma_{\min}$ is a *hard gate*: a candidate must be feasible on every paired seed of every training cell, or it is rejected (its score is set to $-\infty$ and it is excluded from the archive as infeasible). The cross-cell aggregate is *farm-balanced*: the mean over farms of each farm's mean cell score, with a worst-cell tiebreak,

$$\text{fitness} \;=\; \frac{1}{|\mathcal{F}|}\sum_{f\in\mathcal{F}}\frac{1}{|\mathcal{C}_f|}\sum_{c\in\mathcal{C}_f}\text{score}_c, \qquad \text{tiebreak} \;=\; \min_{c}\text{score}_c, \tag{6}$$

where $\mathcal{F}$ is the set of farms and $\mathcal{C}_f$ the scored cells on farm f. Averaging over farms rather than over cells makes each farm contribute equally irrespective of how many cells it carries (the training set has more cells on the large offshore farm than on the small multi-zone farm), so a percent improvement is worth the same on either farm; the worst-cell tiebreak still prevents a large win on one easy cell from masking a regression on a hard one. The aggregate is taken over the *scored* cells. A cell whose objective is saturated — for example a unidirectional resource in which every constraint-feasible layout that separates the turbines reaches free-stream power, so all such layouts score identically — carries no AEP-improvement signal; it is retained as a *feasibility-only* gate (evaluated at two seeds, participating in the hard feasibility gate) but is excluded from the scored set.

2.9 Provenance and lineage

Every candidate is recorded in an append-only lineage log with a content hash, its parent identifier(s), the mutation engine and model string, the token count and monetary cost, its behavioral descriptors, its per-cell fitness, and its generation and island. Seeded ancestors are logged as generation 0 together with their port transforms. Because the seeds and their transforms are recorded, any later claim that the system *discovered* a schedule family is measured against what was seeded, and "discovered" is distinguishable from "recombined a seed."

2.10 Firewall and scoped workspace

The search must not overfit to, or leak, the held-out and test farms. Two disciplines enforce this. First, *information firewall*: only firewall-safe feedback — per-cell percentage scores and feasibility booleans — ever reaches a mutation engine; held-out and test AEP values never enter a mutator's context or transcript, and the pre-registration of the selection and test design (Section 2.11) lives outside the search entirely. Second, *workspace scoping*: before each mutation the engine is run inside a freshly materialized clean-room directory that contains only the schedule interface, a sanitized copy of the skeleton and seed schedules, and the firewall-safe feedback; the results, analysis, held-out state, and pre-registration are outside the engine's working directory and tool scope. Sanitization strips provenance and redacts any residual path token from the reference code, and a gate check refuses to launch if a forbidden token or a syntactically invalid reference file is present in the scoped workspace.

2.11 Selection and deployment criterion

Selection is margin-aware and never a single-evaluation argmax. Among the archive elites that are feasible on the training and held-out farms, the deployed schedule is the one that maximizes the multi-seed validation-mean margin over the reference on the held-out farm, provided that margin exceeds the held-out cell's evaluation-texture floor; if no elite clears the margin, the native seed is deployed and the run is recorded as producing no improvement. The deployment claim is then confirmed, once, on a pre-registered test set under a paired multi-seed protocol (identical per-seed initial layouts across arms) and, as a fidelity stage, re-validated in the full stochastic optimizer. Because feasibility is judged against a tolerance and reference results depend on it, all comparisons are reported at matched tolerance.

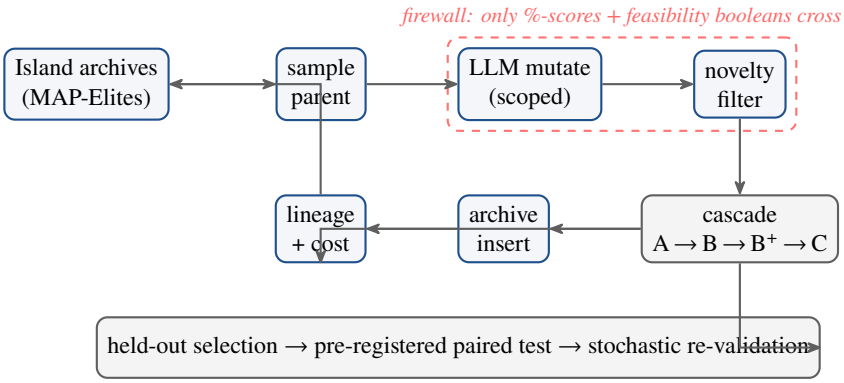

Figure 1: The discovery loop. Parents are sampled from per-island MAP-Elites archives; an LLM mutation engine, run inside a scoped clean-room workspace, proposes a child schedule from the parent source and firewall-safe feedback; a novelty filter rejects near-duplicates before evaluation; the cascade evaluator scores survivors; feasible elites are archived and every candidate is logged with full provenance. Held-out selection, the pre-registered paired test, and stochastic re-validation sit outside the loop.

3 Application

We apply the framework across three wind-farm geometries and four wind resources. The farms span a range of rotor diameters and constraint geometries: the Danish Energy Island (DEI) cluster with IEA 15 MW turbines ($D = 240$ m, single polygon); the IEA Wind 740-10-MW Reference Offshore Wind Plant (ROWP) with IEA 10 MW turbines ($D = 198$ m, single polygon), held out for selection; and the Parque Ficticio inclusion-zone case of Criado Risco et al. (2024) with Vestas V80 turbines ($D = 80$ m, five disconnected inclusion zones — the hardest feasibility geometry). Minimum spacing is four rotor diameters for DEI and ROWP and two rotor diameters for Parque. We consider four wind resources: the measured DEI and ROWP roses, an omnidirectional resource, and a unidirectional resource as limiting cases.

Training cells. The search optimizes and sees percentage feedback on the training cells of Table 2, which span two rotor diameters, turbine counts from 10 to 120, three resource types (the DEI rose, omnidirectional, and unidirectional — the ROWP rose is held out), the multi-zone geometry, and a high-count cell. Two cheap cells serve as the Stage-A fast-reject filter. The reference AEP column is the scale constant of Equation (5), and the floor column is the per-cell texture floor. One Parque cell under the unidirectional resource is a feasibility-only gate: its objective is saturated (Section 2.8), so it is scored for feasibility only and excluded from the aggregate. The two capability-frontier cells (Section 2.7) — a high-count DEI cell and a dense Parque cell under the unidirectional resource, on which even the reference is infeasible — are not training cells; they are elite-tier probes only.

Held-out and test farms. ROWP is an intermediate scale not present in training and is used only for margin-aware selection; its AEP is firewalled from the search. The pre-registered test set — touched exactly once, at the deployment decision — comprises the high-count ROWP cells ($N = 200$ and $N = 300$), the Parque site's real heterogeneous wind resource, and a unidirectional extreme on the held-out ROWP farm.

Evaluation-texture floors. AEP is a mildly discontinuous function of the layout because the wake model masks each turbine's contribution outside a cone; at millimetre-scale perturbations this yields a small texture in re-scored AEP. We measure this floor per farm and set the per-cell minimum meaningful margin accordingly: 0.3 GWh for the large offshore DEI farm, 0.1 GWh for the multi-zone

Table 2: Training cells (all references feasible on every seed). D in metres; Ref. AEP is the rotor-diameter-scaled reference (the scale constant of Equation (5)), a multi-seed feasible mean in GWh; Floor is the per-cell evaluation-texture floor (GWh). "feas.-only" marks the saturated-objective cell that gates feasibility but is excluded from the score aggregate.

Cell	Farm	N	D	Resource	Ref. AEP	Floor
DEI-50	DEI	50	240	DEI rose	5560.4	0.3
DEI-80	DEI	80	240	omnidirectional	8818.1	0.3
DEI-120	DEI	120	240	DEI rose	13030.0	0.3
DEI-50 uni.	DEI	50	240	unidirectional	5598.4	0.3
Parque-20	Parque	20	80	DEI rose	231.5	0.1
Parque-14 uni.	Parque	14	80	unidirectional	*184.8 (f.o.)*	0.1
Parque-10	Parque	10	80	omnidirectional	127.0	0.1

Parque farm, and 0.64 GWh for the held-out ROWP farm. Selection margins use these per-cell floors rather than a single global threshold; the gross Stage-A filter (Section 2.7) does not use them.

4 Reproducibility and cost control

The framework is engineered to be re-runnable and auditable. A content-addressed cache keyed by (schedule hash, cell, seed, $\gamma_{\min}, T$) returns a stored result for any previously computed evaluation, so a resumed run recomputes nothing. State — the archives, the novelty seen-set, the cost tally, and the generation counter — is checkpointed atomically each generation; the parent-sampling randomness is derived from the run identifier and generation, and parent lists are ordered canonically, so a resumed run reproduces the same candidates and hits the cache for every evaluation. The single-precision canonicalization of α_0 (Section 2.4) makes each evaluation *bit-identical across independent processes on a fixed platform*. Across different hardware architectures a small floating-point drift remains — of order ± 1.7 GWh on the large offshore farm between arm64 and x86 in our measurements — so cross-platform results are compared at that measured drift tolerance rather than as bit-identity. A cost ceiling tracks cumulative tokens and spend from the per-invocation engine logs and cleanly aborts the run near the ceiling (it stops issuing new mutations, finishes in-flight work, checkpoints, and stops). Together these make a discovery run a deterministic function of its seeds, configuration, and budget on a fixed platform.

5 Conclusions

We described a framework for discovering deployable wind-farm layout optimizers with large language models. Its distinguishing features are a scale-aware schedule interface that removes the free learning rate and builds the exploration scale from the rotor diameter; a quality-diversity evolutionary loop whose cascade evaluator scores candidates across a multi-farm, multi-scale training suite under a hard feasibility gate; and a provenance, firewall, and reproducibility layer — lineage, a behavioral archive, content-addressed caching, a scoped mutator workspace, and a pre-registered selection and deployment criterion — that make the procedure auditable and repeatable. The framework turns LLM-driven program search from an opportunistic experiment into an engineering procedure whose output is a schedule ready to be deployed and confirmed on unseen farms.

A Schedule interface and mutation prompt

The searched object is a single Python function; the skeleton, seeds, and firewall-safe feedback are provided read-only in the scoped workspace.

```
def schedule_fn(step, total_steps, D, min_spacing,
                n_turbines, gamma_min, alpha0):
    """Return (lr, alpha, beta1, beta2) at `step`.

    D          : rotor diameter (m)            -- exploration-scale source
    min_spacing: minimum inter-turbine spacing (m)
    n_turbines : N                             -- density / packing signal
    gamma_min  : constraint tolerance (m)      -- the terminal learning rate,
                                                  the only user-supplied scale
    alpha0     : penalty normalizer = mean(|grad J|)/D  (skeleton-computed)
    """
    ...
    return lr, alpha, beta1, beta2
```

The mutation engine receives, in its scoped workspace: the interface above, a sanitized copy of the fixed skeleton and the seed schedules, the parent schedule source, and the firewall-safe feedback (per-cell percentage scores and feasibility booleans). It returns one improved module defining `schedule_fn`. No held-out or test AEP, and none of the analysis or pre-registration, is ever placed in that workspace.

References

Bastankhah, M. and Porté-Agel, F.: A new analytical model for wind-turbine wakes, *Renewable Energy*, 70, 116–123, 2014.

Criado Risco, J., Valotta Rodrigues, R., Friis-Møller, M., Quick, J., Mølgaard Pedersen, M., and Réthoré, P.-E.: Gradient-based wind farm layout optimization with inclusion and exclusion zones, *Wind Energy Science*, 9, 585–600, 2024.

Google DeepMind: AlphaEvolve: a coding agent for scientific and algorithmic discovery, Technical report, 2025.

Kingma, D. P. and Ba, J.: Adam: a method for stochastic optimization, in *International Conference on Learning Representations*, 2015.

Mouret, J.-B. and Clune, J.: Illuminating search spaces by mapping elites, arXiv preprint arXiv:1504.04909, 2015.

OpenEvolve: an open-source evolutionary coding framework, software, 2025.

Quick, J., Réthoré, P.-E., Mølgaard Pedersen, M., and Rodrigues, R. V.: Stochastic gradient descent for wind farm optimization, *Wind Energy Science Discussions*, 1–17, 2022.

Romera-Paredes, B., Barekatain, M., Novikov, A., Balog, M., Kumar, M. P., Dupont, E., Ruiz, F. J., Ellenberg, J. S., Wang, P., Fawzi, O., et al.: Mathematical discoveries from program search with large language models, *Nature*, 625, 468–475, 2024.

Sakana AI: ShinkaEvolve: sample-efficient evolutionary program search with large language models, software/technical report, 2025.